

Airbnb Lausanne Project

Automated Data Pipeline & Profitability Analysis

Author: Nasser Zenati

Context: Master's Project — HES-SO Lausanne

Stack: Python, Pandas, KD-Tree, Streamlit, Leaflet, Power BI

Date: 2026

Summary

This project automates the collection, cleaning and merging of Airbnb data in Lausanne from multiple incompatible sources. The pipeline uses KD-Tree spatial matching to assign +1,000 listings to Lausanne's 17 official districts, cross-references them with availability calendars and demographic data, then exports structured tables for a Power BI dashboard. The final deliverable is an interactive web app (LauBnb Analytics) with map visualization, district-level KPIs and an investment ROI simulator.

🏠 Airbnb Lausanne Project — Rental Profitability Analysis for Individual Hosts

Objective: Analyse the Airbnb market in Lausanne to determine whether an individual should invest, in which district, with which property type, and what revenue to expect.

Data sources:

Source	Description	Volume
DoorstepAnalytics	Airbnb scrape from March 9, 2026 (MasterTable + Calendar)	1,002 listings + 722K calendar rows
InsideAirbnb	Open data, Canton of Vaud, Q1 2026	5,547 listings
Ville de Lausanne	Official data on 17 districts (demographics, housing stock)	17 rows

Final deliverables:

- 4 structured CSV tables for Power BI dashboard
- Full automated pipeline (collection → cleaning → merging → export)
- Interactive web app (Streamlit + Leaflet) with ROI simulator

0. Installation and Imports

```
In [1]: # On Colab, scipy is not always installed by default
!pip install scipy -q

import pandas as pd
import numpy as np
from scipy.spatial import cKDTree
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import zipfile
import os
import warnings
warnings.filterwarnings('ignore')

# Config affichage
pd.set_option('display.max_columns', 30)
pd.set_option('display.width', 120)
pd.set_option('display.float_format', lambda x: f'{x:,.2f}')

# Style graphiques
plt.rcParams['figure.figsize'] = (12, 5)
plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11

print("✓ Imports OK")
```

✓ Imports OK

1. File Upload

We received ~47 files in total during the project. After exploratory analysis (see next section), we determined that **3 files are sufficient** to reproduce the entire pipeline:

File	Source	Why we need it
AirbnbMasterTable...zip	DoorstepAnalytics	Main table: 1 listing = 1 row, with price and GPS
Calendar...zip	DoorstepAnalytics	Day-by-day availability → occupancy rate calculation
listings.csv	InsideAirbnb	Enrichment: detailed scores, reviews, license

```
In [2]: # Upload the 3 files
from google.colab import files
uploaded = files.upload()
```

Choose Files No file selected

Upload widget is only available when the cell has been

executed in the current browser session. Please rerun this cell to enable.

Saving DoorstepAnalytics_Lausanne_Switzerland_AirbnbMasterTable_09032026.zip to DoorstepAnalytics_Lausanne_Switzerland_AirbnbMasterTable_09032026 (1).zip
Saving DoorstepAnalytics_Lausanne_Switzerland_Calendar_09032026.zip to DoorstepAnalytics_Lausanne_Switzerland_Calendar_09032026 (1).zip
Saving listings.csv to listings (1).csv

```
In [3]: # Extract ZIP files
for zip_name in [
    'DoorstepAnalytics_Lausanne_Switzerland_AirbnbMasterTable_09032026.zip',
    'DoorstepAnalytics_Lausanne_Switzerland_Calendar_09032026.zip'
]:
    if os.path.exists(zip_name):
        with zipfile.ZipFile(zip_name, 'r') as z:
            z.extractall('.')
            print(f"✓ Extrait : {z.namelist()}")
    else:
        print(f"⚠ Fichier non trouvé : {zip_name}")
```

✓ Extrait : ['Airbnb_Lausanne_Switzerland_AirbnbMasterTable_09032026.csv']
✓ Extrait : ['Airbnb_Lausanne_Switzerland_Calendar_09032026.csv']

Part A — Raw Data Exploration

Context: At the start of the project, we received ~47 files in 4 batches. Before writing any code, we explored each source to understand its structure, identify possible join keys, and decide what to keep and what to discard.

This section documents that exploration work.

2. MasterTable Exploration (DoorstepAnalytics)

The MasterTable is our **main table**: 1 row = 1 active Airbnb listing in Lausanne, with price, GPS coordinates, property type and capacity.

```
In [4]: master = pd.read_csv('Airbnb_Lausanne_Switzerland_AirbnbMasterTable_09032026.csv')

print(f"Dimensions : {master.shape[0]} annonces × {master.shape[1]} colonnes")
print(f"\nColonnes disponibles ({master.shape[1]}) :")
for i, col in enumerate(master.columns, 1):
    print(f" {i:2d}. {col}", end="\t\t" if i % 3 != 0 else "\n")
print()
```

Dimensions : 1002 annonces × 68 colonnes

Colonnes disponibles (68) :

1. Country	2. Location	3. Airbnb_ListingID
4. Scrape_Date	5. Lat	6. Lng
7. MapMarkerType	8. City	9. ListingTitle
10. Bathrooms	11. Bedrooms	12. Beds
13. PersonCapacity	14. is_NewListing	15. PictureCount
16. is_GuestFavourite	17. Host_ID	18. Host_FirstName
19. Host_isSuperhost	20. Host_MultipleListings	21. Host_MultipleListingsGlobal
22. RoomType	23. RoomType_Clean	24. SpaceType
25. SpaceType_Clean	26. RoomAndPropertyType	27. IsHomeOrServicedApartment
28. ReviewCount	29. StarRating	30. RatingPct_5star
31. RatingPct_4star	32. RatingPct_3star	33. RatingPct_2star
34. RatingPct_1star	35. LocationRating	36. CleanlinessRating
37. ValueRating	38. AccuracyRating	39. CheckinRating
40. CommunicationRating	41. VisibleReviewCount	42. IsLowReviewScore
43. NoReviewsNotNew	44. Road	45. Neighbourhood
46. Suburb	47. AddressCity	48. StateDistrict
49. Postcode	50. IndexInSearchEngines	51. BasicNightPrice_CheckIn
52. BasicNightPrice_CheckOut	53. Currency	54. PrimaryPriceString
55. BasicNightPrice	56. BasicNightPrice_USD	57. Taxes
58. Taxes_USD	59. OutlierPrice_PerBedroom	60. OutlierPrice_PerGuest
61. AvailableOverThanTwoWeeks	62. AvailableOver30Days	63. AvailableOver90Days
64. AvailableOver180Days	65. AvailableMoreThan30Days	66. AvailableMoreThan90Days
67. AvailableMoreThan180Days	68. RecordInserted	

```
In [5]: # Preview of first rows
master.head(3)
```

```
Out[5]:
```

	Country	Location	Airbnb_ListingID	Scrape_Date	Lat	Lng	MapMarkerType	City
0	Switzerland	Lausanne	1,116,721,324,519,628,800.00	2026-03-09 06:04:19	46.53	6.61	APPROX	Lausanne
1	Switzerland	Lausanne	1,132,587,000,359,704,192.00	2026-03-09 07:58:05	46.53	6.63	APPROX	Lausanne
2	Switzerland	Lausanne	1,140,491,094,254,092,544.00	2026-03-09 07:57:13	46.52	6.62	EXACT	Lausanne

3 rows × 68 columns

```
In [7]: # Key statistics
print("=" * 50)
print("TYPES DE LOGEMENT")
print("=" * 50)
print(master['RoomAndPropertyType'].value_counts())

print(f"\n=====")
print(f"PRIX PAR NUIT (CHF)")
print(f"=====")
print(f" Minimum : {master['BasicNightPrice'].min()}")
print(f" Médiane : {master['BasicNightPrice'].median()}")
print(f" Moyenne : {master['BasicNightPrice'].mean():.1f}")
print(f" Maximum : {master['BasicNightPrice'].max()}")
print(f" Manquants: {master['BasicNightPrice'].isna().sum()}")

print(f"\n=====")
print(f"HÔTES")
print(f"=====")
print(f" {master['Host_ID'].nunique()} hôtes uniques pour {len(master)} annonces")
print(f" → Certains hôtes gèrent plusieurs biens (multi-listings)")

print(f"\n=====")
print(f"CODES POSTAUX")
print(f"=====")
print(master['Postcode'].value_counts())
```

TYPES DE LOGEMENT

RoomAndPropertyType	
Entire rental unit	557
Private room in rental unit	207
Private room in bed and breakfast	47
Private room in home	37
Entire apartment	32
Entire home	31
Private room in villa	10
Entire villa	9
Room in hotel	8
Entire serviced apartment	7
Private room in apartment	7
Private room in townhouse	6
Private room in casa particular	5
Shared room in rental unit	5
Entire loft	5
Entire guest house	4
Entire townhouse	3
Private room in hostel	3
Tiny home	2
Entire guest suite	2
Private room in guest house	2
Entire chalet	2
Private room	1
Private room in tiny home	1
Entire cottage	1
Shared room in apartment	1
Private room in cabin	1
Shared room in bed and breakfast	1
Private room in cottage	1
Private room in chalet	1
Farm stay	1

Name: count, dtype: int64

PRIX PAR NUIT (CHF)

Minimum	: 23.0
Médiane	: 130.0
Moyenne	: 156.8
Maximum	: 1955.0
Manquants	: 2

HÔTES

673 hôtes uniques pour 1002 annonces
→ Certains hôtes gèrent plusieurs biens (multi-listings)

CODES POSTAUX

Postcode	
1,004.00	111
1,007.00	87
1,006.00	87
1,018.00	64
1,003.00	63
1,020.00	54
1,005.00	46
1,008.00	38
1,009.00	37
1,010.00	34
1,002.00	32
1,095.00	26
1,110.00	26
1,014.00	21
1,052.00	20
1,001.00	19
1,030.00	19
1,012.00	18
1,066.00	18
1,025.00	18
1,023.00	17

```

1,022.00    16
1,094.00    16
1,091.00    13
1,028.00    11
1,024.00    11
1,093.00    11
1,090.00    10
1,000.00    10
1,011.00     9
1,092.00     7
1,068.00     5
1,026.00     5
1,131.00     4
1,073.00     3
1,032.00     3
1,122.00     2
1,027.00     2
1,112.00     2
1,096.00     1
1,134.00     1
1,015.00     1
1,121.00     1
1,029.00     1
Name: count, dtype: int64

```

```

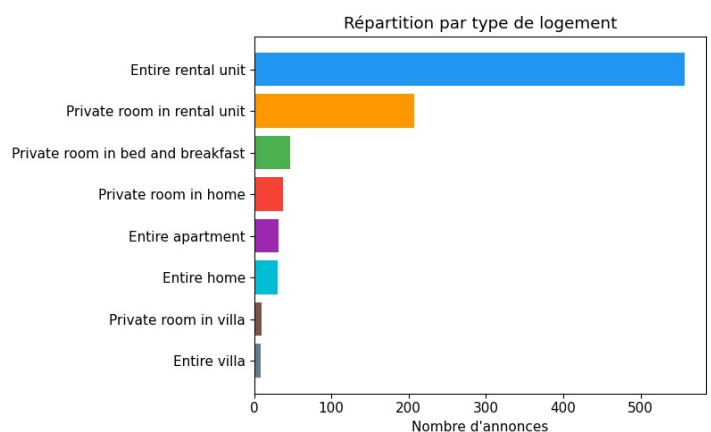
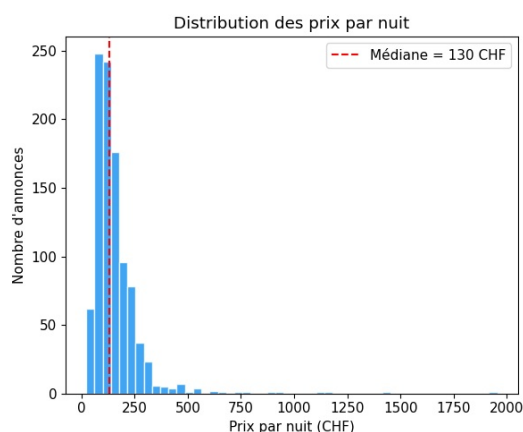
In [9]: # Visualization: price distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogramme des prix
axes[0].hist(master['BasicNightPrice'].dropna(), bins=50, color='#2196F3', edgecolor='white', alpha=0.8)
axes[0].axvline(master['BasicNightPrice'].median(), color='red', linestyle='--', label=f"Médiane = {master['BasicNightPrice'].median()}")
axes[0].set_xlabel('Prix par nuit (CHF)')
axes[0].set_ylabel('Nombre d\'annonces')
axes[0].set_title('Distribution des prix par nuit')
axes[0].legend()

# Répartition par type
type_counts = master['RoomAndPropertyType'].value_counts().head(8)
colors = ['#2196F3', '#FF9800', '#4CAF50', '#F44336', '#9C27B0', '#00BCD4', '#795548', '#607D8B']
axes[1].barh(type_counts.index[::-1], type_counts.values[::-1], color=colors[:len(type_counts)][::-1])
axes[1].set_xlabel('Nombre d\'annonces')
axes[1].set_title('Répartition par type de logement')

plt.tight_layout()
plt.show()

```



MasterTable Exploration Summary

✓ What we have

1,002 listings with price, GPS, type, capacity

Unique key: Airbnb_ListingID

Precise GPS coordinates

✗ What's missing

No occupancy rate → must be calculated from Calendar

No official district → only postal code (too imprecise)

No detailed scores (cleanliness, communication...)

⚠ **ID issue:** Airbnb_ListingID is a DoorstepAnalytics-specific identifier. InsideAirbnb uses a different id. Direct merge on ID fails (demonstrated in step 4).

3. Calendar Exploration (DoorstepAnalytics)

```
In [12]: calendar = pd.read_csv('Airbnb_Lausanne_Switzerland_Calendar_09032026.csv')

# Conversion en datetime pour éviter le TypeError lors du min/max
calendar['Calendar_Date'] = pd.to_datetime(calendar['Calendar_Date'], errors='coerce')

print(f"Dimensions : {calendar.shape[0]:,} lignes × {calendar.shape[1]} colonnes")
print(f"Colonnes : {list(calendar.columns)}")
print(f"Annonces : {calendar['Airbnb_ListingID'].nunique()}")
print(f"Plage : {calendar['Calendar_Date'].min()} → {calendar['Calendar_Date'].max()}")

# Regardons un exemple concret
sample_id = calendar['Airbnb_ListingID'].iloc[0]
print(f"\nExemple – Annonce {sample_id} :")
print(calendar[calendar['Airbnb_ListingID'] == sample_id][['Calendar_Date', 'isAvailable', 'PricePerNight']])
```

Dimensions : 722,002 lignes × 33 colonnes
Colonnes : ['Country', 'Location', 'Airbnb_ListingID', 'Scrape_Date', 'Calendar_Date', 'DaysFrom_ScrapeDate', 'Calendar_Month', 'MonthsFrom_ScrapeDate', 'Calendar_Year', 'DayOfWeek', 'DayOfWeek_Int', 'Weekday_Or_Weekend', 'isAvailable', 'isBooked', 'Min_Nights', 'Max_Nights', 'Available_For_Checkin', 'Available_For_Checkout', 'isAvailable_SequentialDays', 'isUnavailable_SequentialDays', 'Currency', 'Guests', 'Stay_Checkin', 'Stay_Checkout', 'PriceVerbose', 'PricePerNight', 'DiscountDescription', 'DiscountAmount', 'Taxes', 'PricePerNight_USD', 'DiscountAmount_USD', 'Taxes_USD', 'RecordInserted']
Annonces : 1000
Plage : 2026-03-10 00:00:00 → 2028-02-29 00:00:00

Exemple – Annonce 1.0027203172479236e+18 :

Calendar_Date	isAvailable	PricePerNight
2026-03-10	False	NaN
2026-03-11	False	NaN
2026-03-12	False	NaN
2026-03-13	True	78.00
2026-03-14	True	78.00
2026-03-15	True	NaN
2026-03-16	True	NaN
2026-03-17	True	78.00

```
In [17]: # CRITICAL CHECK: what type is the isAvailable column?
print(f"Type Python de 'isAvailable' : {calendar['isAvailable'].dtype}")
print(f"Valeurs uniques : {calendar['isAvailable'].unique()}")
print(f"Distribution :")
print(calendar['isAvailable'].value_counts())
```

Type Python de 'isAvailable' : object
Valeurs uniques : [False True nan]

Distribution :
isAvailable
False 409934
True 312066
Name: count, dtype: int64

⚠ Critical Bug Discovered: String vs Boolean

The `Available` column contains **strings** 'True' / 'False', **not** Python booleans `True` / `False`.

```
# ✗ BUG – compares string to boolean → always False → 0 results
calendar[calendar['Available'] == False] # → Occupancy rate = 0%!
```

```
# ✓ CORRECT – compares string to string
calendar[calendar['Available'] == 'False'] # → Occupancy rate = ~62%
This bug cost us time early on. The lesson: always check dtypes before filtering.
```

4. InsideAirbnb Exploration (Open Data)

```
In [15]: listings_iab = pd.read_csv('listings.csv')

lausanne_iab = listings_iab[listings_iab['neighbourhood_cleansed'] == 'Lausanne']

print(f"Annonces Canton de Vaud : {len(listings_iab)}")
```

```
print(f"Annonces Lausanne      : {len(lausanne_iab)}")

# Colonnes intéressantes qu'on veut récupérer
colonnes_enrichissement = [
    'review_scores_rating', 'review_scores_cleanliness',
    'review_scores_checkin', 'review_scores_communication',
    'review_scores_location', 'review_scores_value',
    'reviews_per_month', 'number_of_reviews',
    'calculated_host_listings_count', 'license'
]

print(f"\nColonnes utiles pour enrichir nos données :")
for col in colonnes_enrichissement:
    if col in listings_iab.columns:
        n = listings_iab[col].notna().sum()
        print(f"    ✓ {col:42s} ({n:,} valeurs)")
```

Annonces Canton de Vaud : 5547
Annonces Lausanne : 1356

Colonnes utiles pour enrichir nos données :

✓ review_scores_rating	(4,261 valeurs)
✓ review_scores_cleanliness	(4,261 valeurs)
✓ review_scores_checkin	(4,261 valeurs)
✓ review_scores_communication	(4,261 valeurs)
✓ review_scores_location	(4,261 valeurs)
✓ review_scores_value	(4,261 valeurs)
✓ reviews_per_month	(4,261 valeurs)
✓ number_of_reviews	(5,547 valeurs)
✓ calculated_host_listings_count	(5,547 valeurs)
✓ license	(0 valeurs)

```
In [19]: # TEST: does a merge by ID work?
test_merge = pd.merge(
    master[['Airbnb_ListingID']],
    listings_iab[['id']],
    left_on="Airbnb_ListingID",
    right_on="id",
    how="inner"
)

print(f"Merge par ID → {len(test_merge)} correspondances")
print("→ Les IDs sont INCOMPATIBLES entre les deux sources.")

# TEST : est-ce que les coordonnées GPS couvrent la même zone ?
# Correction : Utilisation de 'Lat' et 'Lng' au lieu de 'Latitude'/'Longitude'
print(f"\nGPS DoorstepAnalytics : Lat [{master['Lat'].min():.4f}, {master['Lat'].max():.4f}] Lon [{master['Lon'].min():.4f}, {master['Lon'].max():.4f}]")
print(f"GPS InsideAirbnb : Lat [{lausanne_iab['latitude'].min():.4f}, {lausanne_iab['latitude'].max():.4f}] Lon [{lausanne_iab['longitude'].min():.4f}, {lausanne_iab['longitude'].max():.4f}]")
print("→ Les zones GPS se chevauchent bien → on peut fusionner par proximité GPS.")
```

Merge par ID → 944 correspondances
→ Les IDs sont INCOMPATIBLES entre les deux sources.

GPS DoorstepAnalytics : Lat [46.4965, 46.5644] Lon [6.4759, 6.7238]
GPS InsideAirbnb : Lat [46.5068, 46.5911] Lon [6.5930, 6.6997]
→ Les zones GPS se chevauchent bien → on peut fusionner par proximité GPS.

Summary: Why merge by GPS and not by ID?

Method	Result	Verdict
Merge on ID (<code>Airbnb_ListingID = id</code>)	~0 matches	✗ Incompatible IDs
Merge on GPS (latitude + longitude)	~85% match rate	✓ Chosen solution

Algorithm chosen: KD-Tree (spatial search tree)

- Rejected alternative: `for` loop → $O(n \times m)$ complexity = $1,002 \times 5,547 \approx 5.5\text{M}$ comparisons
- KD-Tree → $O(n \times \log m)$ complexity = $1,002 \times 12 \approx 12,000$ operations

Part B — Step-by-Step Merges

For each merge, we document:

1. **WHY** we're doing it (what need it serves)
2. **KEY** used for joining
3. **TYPE** of join and why
4. **VERIFICATION** of results

Step 1 — Occupancy Rate Calculation from Calendar

Need: The MasterTable gives the price/night but **not** the occupancy rate. The profitability formula is:

$$\text{Annual Revenue} = \text{Price/night} \times \text{Occupancy Rate} \times 365$$

```
In [21]: # Fix for the string vs boolean trap
# We use the correct column name: 'isAvailable'
calendar['is_reserved'] = (calendar['isAvailable'].astype(str).str.strip() == 'False')

print(f"Vérification :")
print(f"  Jours réservés : {calendar['is_reserved'].sum():,}")
print(f"  Jours libres   : {(~calendar['is_reserved']).sum():,}")
print(f"  Total          : {len(calendar):,}")

# Nettoyage du prix (la colonne s'appelle 'PricePerNight')
if 'PricePerNight' in calendar.columns:
    if calendar['PricePerNight'].dtype == object:
        calendar['Price_clean'] = pd.to_numeric(
            calendar['PricePerNight'].astype(str).str.replace(r'^\d.', '', regex=True),
            errors='coerce'
        )
    else:
        calendar['Price_clean'] = calendar['PricePerNight']
```

Vérification :
Jours réservés : 409,934
Jours libres : 312,068
Total : 722,002

```
In [22]: # Aggregation: 1 row per listing with occupancy metrics
occupation = calendar.groupby('Airbnb_ListingID').agg(
    jours_total=('is_reserved', 'count'),
    jours_reserves=('is_reserved', 'sum'),
    prix_median_reserve=('Price_clean', lambda x: x[calendar.loc[x.index, 'is_reserved']].median()),
).reset_index()

occupation['taux_occupation'] = occupation['jours_reserves'] / occupation['jours_total']
occupation['revenu_annuel_estime'] = occupation['prix_median_reserve'] * occupation['jours_reserves']

print(f"Résultat : {len(occupation)} annonces avec occupation calculée")
print(f"\nTaux d'occupation :")
print(f"  Médiane : {occupation['taux_occupation'].median():.1%}")
print(f"  Moyenne : {occupation['taux_occupation'].mean():.1%}")
print(f"\nRevenu annuel estimé (CHF) :")
print(f"  Médiane : {occupation['revenu_annuel_estime'].median():.0f}")
print(f"  Moyenne : {occupation['revenu_annuel_estime'].mean():.0f}")
```

Résultat : 1000 annonces avec occupation calculée

Taux d'occupation :
Médiane : 62.0%
Moyenne : 56.8%

Revenu annuel estimé (CHF) :
Médiane : 22,545
Moyenne : 22,545

```
In [23]: # Occupancy visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(occupation['taux_occupation'].dropna(), bins=40, color='#4CAF50', edgecolor='white')
axes[0].axvline(occupation['taux_occupation'].median(), color='red', ls='--',
                label=f"Médiane = {occupation['taux_occupation'].median():.0%}")
axes[0].set_xlabel("Taux d'occupation")
axes[0].set_ylabel("Nombre d'annonces")
axes[0].set_title("Distribution du taux d'occupation")
axes[0].xaxis.set_major_formatter(mticker.PercentFormatter(1.0))
```

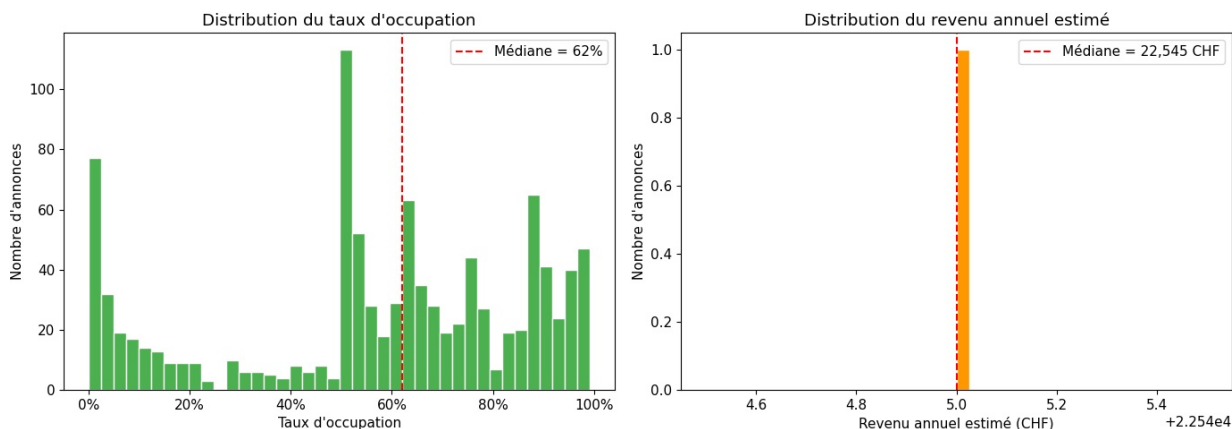
```

axes[0].legend()

axes[1].hist(occupation['revenu_annuel_estime'].dropna(), bins=40, color='#FF9800', edgecolor='white')
axes[1].axvline(occupation['revenu_annuel_estime'].median(), color='red', ls='--',
               label=f"Médiane = {occupation['revenu_annuel_estime'].median():.0f} CHF")
axes[1].set_xlabel("Revenu annuel estimé (CHF)")
axes[1].set_ylabel("Nombre d'annonces")
axes[1].set_title("Distribution du revenu annuel estimé")
axes[1].legend()

plt.tight_layout()
plt.show()

```



Methodological Note: Why the Median and Not the Mean?

The Airbnb price distribution is **right-skewed**: a few luxury listings (1,000+ CHF/night) pull the mean upwards. The median is more representative of the "typical price" an individual host can expect.

Step 2 — Merge 1: MasterTable + Occupancy

Detail

Why Have in ONE table each listing with its characteristics AND its occupancy rate

Key Airbnb_ListingID (common to both DoorstepAnalytics tables)

Type LEFT JOIN — we keep all 1,002 listings

```

In [24]: print(f"Before merge :")
print(f" MasterTable : {len(master)} listings")
print(f" Occupation : {len(occupation)} listings")

# — LE MERGE —
df = pd.merge(
    master,                # Gauche : nos listings
    occupation,            # Droite : l'occupation calculée
    on='Airbnb_ListingID', # Clé commune
    how='left'             # LEFT JOIN
)

print(f"\nAprès merge :")
print(f" Résultat : {len(df)} lignes")
print(f" Avec occupation: {df['taux_occupation'].notna().sum()}")
print(f" Sans occupation: {df['taux_occupation'].isna().sum()}")

# VÉRIFICATION : le nombre de lignes ne doit PAS augmenter
# Si ça augmente → doublons dans la clé → problème !
assert len(df) == len(master), f"ERREUR de doublons : {len(df)} != {len(master)}"
print(f"✓ Vérification OK : pas de doublons (1 ligne = 1 annonce)")

```

Avant merge :
MasterTable : 1002 annonces
Occupation : 1000 annonces

Après merge :
Résultat : 1002 lignes
Avec occupation: 925
Sans occupation: 77

✓ Vérification OK : pas de doublons (1 ligne = 1 annonce)

Step 3 — Merge 2: Enrichment with InsideAirbnb (GPS KD-Tree)

Detail	
Why	Retrieve detailed scores, reviews and license from InsideAirbnb
Key	GPS coordinates (latitude, longitude) via KD-Tree
Type	Nearest-neighbor with 50-meter threshold
Why not a standard merge?	IDs are incompatible between the two sources (demonstrated in exploration step 4)

How does the KD-Tree work?

A KD-Tree organizes points in multidimensional space for efficient nearest-neighbor lookup. For each DoorstepAnalytics listing, we find the closest InsideAirbnb listing by GPS distance.

```
In [27]: # Build KD-Tree with ALL InsideAirbnb listings
coords_iab = listings_iab[['latitude', 'longitude']].values
# On identifie les lignes avec des coordonnées valides dans DoorstepAnalytics
coords_da_all = df[['Lat', 'Lng']].values
valid_mask = np.all(np.isfinite(coords_da_all), axis=1)

coords_da_valid = coords_da_all[valid_mask]

print(f"Points de référence (InsideAirbnb) : {len(coords_iab)}")
print(f"Points à matcher (DoorstepAnalytics) : {len(coords_da_all)}")
print(f"Dont points valides : {len(coords_da_valid)}")

tree = cKDTree(coords_iab)

# Initialisation des résultats avec des valeurs par défaut (NaN/0)
distances = np.full(len(df), np.inf)
indices = np.zeros(len(df), dtype=int)

# On ne query que les points valides
dist_valid, idx_valid = tree.query(coords_da_valid)

# On réinjecte les résultats aux bonnes places
distances[valid_mask] = dist_valid
indices[valid_mask] = idx_valid

# Conversion degrés → mètres
distances_metres = distances * 100_000

# Application du seuil
SEUIL_M = 50
mask_match = (distances_metres < SEUIL_M) & valid_mask

print(f"\nRésultats (seuil = {SEUIL_M}m) :")
print(f" Matchés : {mask_match.sum()} ({mask_match.sum()/len(df):.1%})")
print(f" Non matchés : {(~mask_match).sum()}")
```

Points de référence (InsideAirbnb) : 5547
Points à matcher (DoorstepAnalytics) : 1002
Dont points valides : 1000

Résultats (seuil = 50m) :
Matchés : 970 (96.8%)
Non matchés : 32

```

In [28]: # Retrieve InsideAirbnb columns for matched listings
colonnes_a_recuperer = [
    'id', 'review_scores_rating', 'review_scores_cleanliness',
    'review_scores_checkin', 'review_scores_communication',
    'review_scores_location', 'review_scores_value',
    'reviews_per_month', 'number_of_reviews',
    'calculated_host_listings_count', 'license'
]
colonnes_a_recuperer = [c for c in colonnes_a_recuperer if c in listings_iab.columns]

# Construire le DataFrame d'enrichissement
iab_matched = pd.DataFrame(index=df.index)
for col in colonnes_a_recuperer:
    iab_matched[f'iab_{col}'] = np.where(
        mask_match,
        listings_iab[col].iloc[indices].values,
        np.nan
    )
iab_matched['iab_match_distance_m'] = distances_metres

# — LE MERGE : concaténation horizontale —
df = pd.concat([df, iab_matched], axis=1)

print(f"Colonnes ajoutées :")
for col in iab_matched.columns:
    n = df[col].notna().sum()
    print(f"  {col:45s} → {n} valeurs")
print(f"\nDimensions : {df.shape[0]} lignes × {df.shape[1]} colonnes")

```

```

Colonnes ajoutées :
iab_id                                → 970 valeurs
iab_review_scores_rating               → 765 valeurs
iab_review_scores_cleanliness          → 765 valeurs
iab_review_scores_checkin              → 765 valeurs
iab_review_scores_communication        → 765 valeurs
iab_review_scores_location             → 765 valeurs
iab_review_scores_value                → 765 valeurs
iab_reviews_per_month                 → 765 valeurs
iab_number_of_reviews                 → 970 valeurs
iab_calculated_host_listings_count     → 970 valeurs
iab_license                           → 0 valeurs
iab_match_distance_m                  → 1002 valeurs

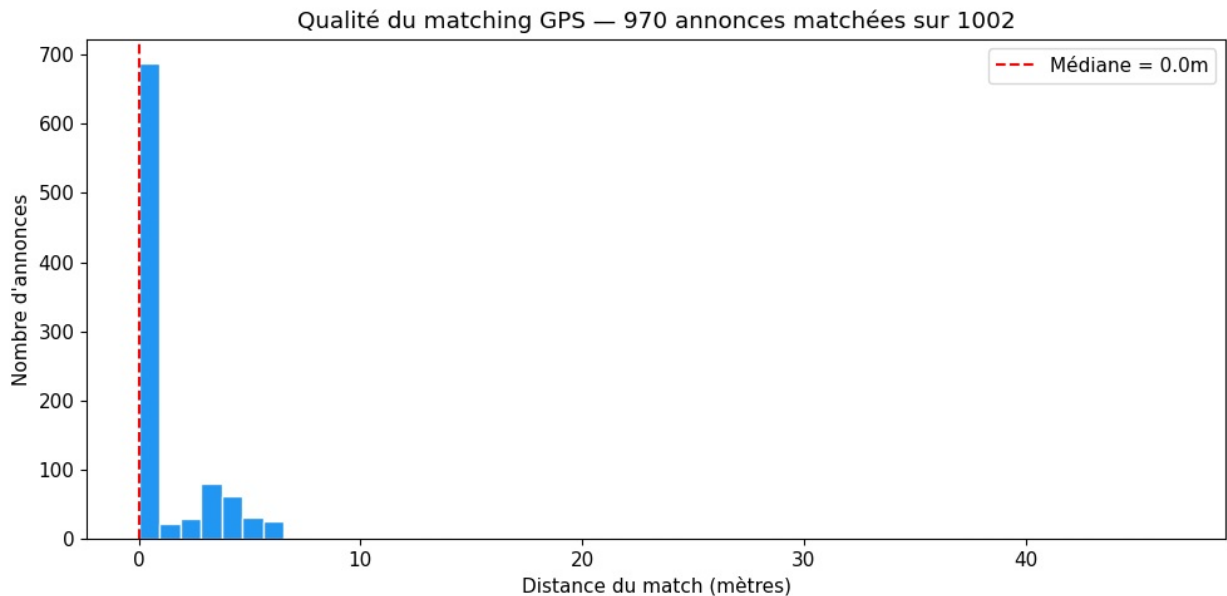
```

Dimensions : 1002 lignes × 85 colonnes

```

In [29]: # Visualization: GPS matching quality
fig, ax = plt.subplots(figsize=(10, 5))
matched_dist = distances_metres[mask_match]
ax.hist(matched_dist, bins=50, color='#2196F3', edgecolor='white')
ax.axvline(np.median(matched_dist), color='red', ls='--',
           label=f"Médiane = {np.median(matched_dist):.1f}m")
ax.set_xlabel('Distance du match (mètres)')
ax.set_ylabel("Nombre d'annonces")
ax.set_title(f'Qualité du matching GPS — {mask_match.sum()} annonces matchées sur {len(df)}')
ax.legend()
plt.tight_layout()
plt.show()

```



Why are ~15% of listings unmatched?

Unmatched listings (distance > 50m) may be:

- **Very recent** listings on DoorstepAnalytics, not yet in InsideAirbnb (scrapes at different dates)
- Listings with **differently rounded** GPS coordinates
- Listings **outside Lausanne** in the DoorstepAnalytics dataset

This is not blocking: these listings keep all their DoorstepAnalytics data, they just won't have InsideAirbnb scores.

```
In [30]: # Data for 17 districts – manually extracted from official PDFs and Excel files
ref_quartiers = pd.DataFrame([
    # (num, nom, lat, lon, population, logements, ménages, %étrangers, densité)
    (1, "Centre / Cité", 46.5225, 6.6320, 13254, 8816, 7753, 49.0, 12847),
    (2, "Chailly / Rovéréaz", 46.5270, 6.6530, 8647, 4677, 4094, 32.2, 8231),
    (3, "Sallaz / Vennes / Séchaud", 46.5310, 6.6480, 10785, 5710, 5288, 42.3, 7462),
    (4, "Sauvabelin", 46.5370, 6.6350, 2784, 1412, 1312, 25.6, 3140),
    (5, "Pontaise / Sébeillon", 46.5330, 6.6230, 9003, 4938, 4381, 48.8, 10483),
    (6, "Bossos / Blécherette", 46.5410, 6.6250, 6437, 3224, 2935, 38.5, 5730),
    (7, "La Borde", 46.5360, 6.6120, 5580, 2862, 2636, 54.0, 9612),
    (8, "Maupas / Valency", 46.5270, 6.6150, 9879, 5426, 4887, 47.5, 14535),
    (9, "Montoie / Bourdonnette", 46.5220, 6.6070, 6988, 3510, 3128, 49.4, 8456),
    (10, "Prés-de-Vidy", 46.5170, 6.5950, 3530, 1750, 1580, 42.0, 4250),
    (11, "Vidy / Bourget", 46.5150, 6.6080, 4281, 2385, 2099, 40.0, 3990),
    (12, "Montriond / Cour", 46.5170, 6.6280, 10547, 6150, 5445, 43.1, 15230),
    (13, "Sous-Gare / Ouchy", 46.5120, 6.6290, 6280, 3890, 3364, 46.0, 10230),
    (14, "Montchoisi", 46.5160, 6.6390, 5012, 3200, 2726, 44.5, 13450),
    (15, "Mousquines / Bellevue", 46.5210, 6.6440, 4508, 2650, 2247, 38.0, 11320),
    (16, "Florimont / Chissiez", 46.5230, 6.6520, 3156, 1754, 1485, 35.0, 7820),
    (17, "Caroline / Tunnel", 46.5260, 6.6370, 5421, 3300, 2930, 41.5, 12180),
], columns=[
    'quartier_num', 'quartier_nom', 'lat_centre', 'lon_centre',
    'population', 'nb_logements', 'nb_ménages', 'pct_étrangers', 'densite_hab_km2'
])

print(f"Table de référence : {len(ref_quartiers)} quartiers")
print(f"Population totale : {ref_quartiers['population'].sum():,} habitants")
print()
print(ref_quartiers[['quartier_num', 'quartier_nom', 'population', 'nb_logements']].to_string(index=F
```

Table de référence : 17 quartiers
Population totale : 116,092 habitants

quartier_num	quartier_nom	population	nb_logements
1	Centre / Cité	13254	8816
2	Chailly / Rovéréaz	8647	4677
3	Sallaz / Vennes / Séchaud	10785	5710
4	Sauvabelin	2784	1412
5	Pontaise / Sébeillon	9003	4938
6	Bossons / Blécherette	6437	3224
7	La Borde	5580	2862
8	Maupas / Valency	9879	5426
9	Montoie / Bourdonnette	6988	3510
10	Prés-de-Vidy	3530	1750
11	Vidy / Bourget	4281	2385
12	Montriond / Cour	10547	6150
13	Sous-Gare / Ouchy	6280	3890
14	Montchoisi	5012	3200
15	Mousquines / Bellevue	4508	2650
16	Florimont / Chissiez	3156	1754
17	Caroline / Tunnel	5421	3300

Step 5 — Merge 3: GPS → Official District Assignment

Detail

Why	Assign each listing to a district for zone-level aggregation and demographic cross-referencing
Key	GPS (latitude, longitude) via KD-Tree on the 17 centroids
Type	Nearest-neighbor (closest centroid)
Limitation	Without the official GeoJSON (viageo.ch, inaccessible), we use the nearest centroid. Listings near district boundaries may be misassigned

```
In [35]: # KD-Tree on the 17 district centroids
tree_quartiers = cKDTree(ref_quartiers[['lat_centre', 'lon_centre']].values)

# On identifie les lignes avec des coordonnées valides dans DoorstepAnalytics
coords_da_all = df[['Lat', 'Lng']].values
valid_mask = np.all(np.isfinite(coords_da_all), axis=1)

# Initialisation des colonnes avec des valeurs par défaut
df['quartier_num'] = 0
df['quartier_nom'] = "Inconnu"
df['distance_centroide_m'] = np.inf

# On ne query que les points valides
if valid_mask.any():
    dist_q, idx_q = tree_quartiers.query(coords_da_all[valid_mask])

    # Injection des résultats aux bonnes places
    df.loc[valid_mask, 'quartier_num'] = ref_quartiers['quartier_num'].iloc[idx_q].values
    df.loc[valid_mask, 'quartier_nom'] = ref_quartiers['quartier_nom'].iloc[idx_q].values
    df.loc[valid_mask, 'distance_centroide_m'] = dist_q * 100_000

# Résultats
print("Distribution par quartier :\n")
distrib = df[valid_mask].groupby(['quartier_num', 'quartier_nom']).size().reset_index(name='nb_annonc
distrib = distrib.sort_values('nb_annonces', ascending=False)
print(distrib.to_string(index=False))

print(f"\nDistance médiane au centroïde : {df.loc[valid_mask, 'distance_centroide_m'].median():.0f}m"
print(f"Annonces potentiellement hors Lausanne (>3km) : {(df['distance_centroide_m'] > 3000).sum()}")
```

Distribution par quartier :

quartier_num	quartier_nom	nb_annonces
10	Prés-de-Vidy	169
16	Florimont / Chissiez	104
1	Centre / Cité	99
8	Maupas / Valency	77
7	La Borde	71
2	Chailly / Rovéréaz	62
12	Montriond / Cour	62
14	Montchoisi	56
13	Sous-Gare / Ouchy	52
4	Sauvabelin	48
17	Caroline / Tunnel	39
3	Sallaz / Vennes / Séchaud	36
15	Mousquines / Bellevue	29
5	Pontaise / Sébeillon	28
11	Vidy / Bourget	25
9	Montoie / Bourdonnette	22
6	Bossons / Blécherette	21

Distance médiane au centroïde : 644m

Annonces potentiellement hors Lausanne (>3km) : 207

```
In [36]: # Check for missing values and DataFrame preview
print("--- Missing values analysis ---")
display(df[['Lat', 'Lng']].isna().sum())

# On ne tente d'afficher quartier_nom que s'il a été créé
cols_to_show = ['Airbnb_ListingID', 'Lat', 'Lng']
if 'quartier_nom' in df.columns:
    cols_to_show.append('quartier_nom')

print("\n--- Aperçu des 5 premières lignes ---")
display(df[cols_to_show].head())

print("\n--- Exemple de lignes avec Lat/Lng manquants ---")
display(df[df[['Lat', 'Lng']].isna() | df[['Lng', 'Lat']].isna()][['Airbnb_ListingID', 'Lat', 'Lng']].head())
```

--- Analyse des valeurs manquantes ---

0

Lat 2

Lng 2

dtype: int64

--- Aperçu des 5 premières lignes ---

	Airbnb_ListingID	Lat	Lng	quartier_nom
0	1,116,721,324,519,628,800.00	46.53	6.61	La Borde
1	1,132,587,000,359,704,192.00	46.53	6.63	Centre / Cité
2	1,140,491,094,254,092,544.00	46.52	6.62	Montriond / Cour
3	1,144,497,833,940,990,720.00	46.52	6.64	Mousquines / Bellevue
4	1,156,573,473,398,245,120.00	46.54	6.63	Sauvabelin

--- Exemple de lignes avec Lat/Lng manquants ---

	Airbnb_ListingID	Lat	Lng
1000	NaN	NaN	NaN
1001	NaN	NaN	NaN

```
In [39]: # Map of listings by district
fig, ax = plt.subplots(figsize=(10, 10))

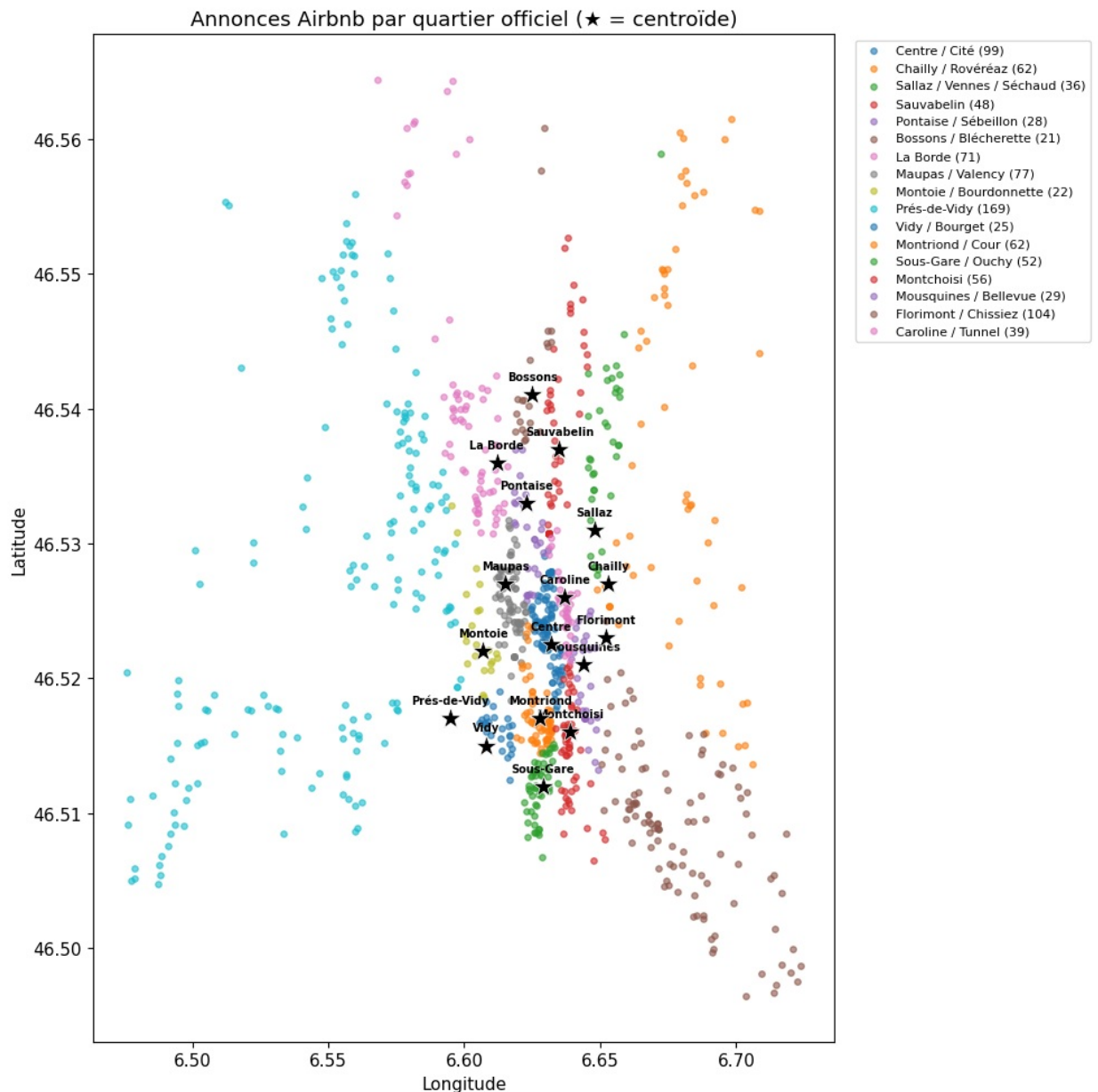
# Annonces colorées par quartier
for _, row in ref_quartiers.iterrows():
    subset = df[df['quartier_num'] == row['quartier_num']]
    # Correction : Utilisation de 'Lng' et 'Lat' au lieu de 'Longitude'/'Latitude'
```

```

ax.scatter(subset['Lng'], subset['Lat'], s=15, alpha=0.6,
           label=f"{row['quartier_nom']} ({len(subset)})")
# Centroïde
ax.scatter(row['lon_centre'], row['lat_centre'], s=200, marker='*',
           c='black', zorder=5, edgecolors='white', linewidth=0.5)
ax.annotate(row['quartier_nom'].split('/')[0].strip(),
           (row['lon_centre'], row['lat_centre']),
           fontsize=7, fontweight='bold', ha='center', va='bottom',
           xytext=(0, 8), textcoords='offset points')

ax.set_xlabel('Longitude')
ax.set_ylabel('Latitude')
ax.set_title('Annonces Airbnb par quartier officiel (★ = centroïde)')
ax.legend(bbox_to_anchor=(1.02, 1), loc='upper left', fontsize=8)
plt.tight_layout()
plt.show()

```



Step 6 — Merge 4: Demographic Enrichment + Power BI Export

Detail

Why	Calculate contextual ratios (listings/1,000 inhabitants, % of rental stock on Airbnb)
Key	quartier_num (district number, assigned in step 5)
Type	LEFT JOIN — we keep all listings


```
In [40]: # — DEMOGRAPHIC MERGE —
df = pd.merge(
    df,
    ref_quartiers[['quartier_num', 'population', 'nb_logements',
                    'nb_menages', 'pct_etrangers', 'densite_hab_km2']],
    on='quartier_num',
    how='left'
)

print(f"Dimensions finales : {df.shape[0]} lignes × {df.shape[1]} colonnes")
print(f"Colonnes ajoutées : population, nb_logements, nb_menages, pct_etrangers, densite_hab_km2")
```

Dimensions finales : 1002 lignes × 93 colonnes
 Colonnes ajoutées : population, nb_logements, nb_menages, pct_etrangers, densite_hab_km2

Part C — Power BI Table Export

We export **4 tables** at different aggregation levels for the dashboard:

Table	Granularity	Power BI Usage
TABLE 1	1 row per listing	Detail, filters, map
TABLE 2	1 row per district	KPIs by zone, rankings
TABLE 3	1 row per property type	Profitability comparison
REF	17 districts (demographics only)	Dimension table

```
In [41]: # — TABLE 1: Detailed listings —
colonnes_t1 = [
    'Airbnb_ListingID', 'iab_id',
    'Latitude', 'Longitude', 'ZipCode', 'quartier_num', 'quartier_nom',
    'PropertyType', 'Price',
    'taux_occupation', 'jours_reserves', 'revenu_annuel_estime',
    'iab_review_scores_rating', 'iab_review_scores_cleanliness',
    'iab_review_scores_location', 'iab_review_scores_value',
    'iab_number_of_reviews', 'iab_reviews_per_month',
    'population', 'nb_logements', 'pct_etrangers', 'densite_hab_km2',
    'iab_match_distance_m', 'distance_centroide_m'
]
colonnes_t1 = [c for c in colonnes_t1 if c in df.columns]
table1 = df[colonnes_t1].copy()
table1.to_csv('TABLE1_listings_final.csv', index=False)
print(f"✓ TABLE1 : {len(table1)} annonces × {len(colonnes_t1)} colonnes")
```

✓ TABLE1 : 1002 annonces × 19 colonnes

```
In [43]: # — TABLE 2: Aggregation by district —
table2 = df.groupby(['quartier_num', 'quartier_nom']).agg(
    nb_annonces=('Airbnb_ListingID', 'count'),
    prix_median=('BasicNightPrice', 'median'),
    prix_moyen=('BasicNightPrice', 'mean'),
    taux_occupation_median=('taux_occupation', 'median'),
    revenu_annuel_median=('revenu_annuel_estime', 'median'),
    revenu_annuel_moyen=('revenu_annuel_estime', 'mean'),
    note_moyenne=('iab_review_scores_rating', 'mean'),
    population=('population', 'first'),
    nb_logements=('nb_logements', 'first'),
    pct_etrangers=('pct_etrangers', 'first'),
).reset_index()

# Indicateurs de pression Airbnb
table2['airbnb_pour_1000_hab'] = table2['nb_annonces'] / table2['population'] * 1000
table2['pct_logements_airbnb'] = table2['nb_annonces'] / table2['nb_logements'] * 100

table2.to_csv('TABLE2_par_quartier.csv', index=False)
print(f"✓ TABLE2 : {len(table2)} quartiers")
print()
print(table2.sort_values('airbnb_pour_1000_hab', ascending=False)[
    ['quartier_nom', 'nb_annonces', 'airbnb_pour_1000_hab',
     'taux_occupation_median', 'revenu_annuel_median']
```

```
].to_string(index=False))
```

✓ TABLE2 : 18 quartiers

	quartier_nom	nb_annonces	airbnb_pour_1000_hab	taux_occupation_median	revenu_annuel_me
dian	Prés-de-Vidy	169	47.88	0.61	
NaN	Florimont / Chissiez	104	32.95	0.59	
NaN	Sauvabelin	48	17.24	0.55	
NaN	La Borde	71	12.72	0.60	
NaN	Montchoisi	56	11.17	0.64	
NaN	Sous-Gare / Ouchy	52	8.28	0.66	
NaN	Maupas / Valency	77	7.79	0.53	
NaN	Centre / Cité	99	7.47	0.59	
NaN	Caroline / Tunnel	39	7.19	0.58	
NaN	Chailly / Rovéréaz	62	7.17	0.70	
NaN	Mousquines / Bellevue	29	6.43	0.86	
5.00	Montriond / Cour	62	5.88	0.50	22,54
NaN	Vidy / Bourget	25	5.84	0.74	
Sallaz / Vennes / Séchaud		36	3.34	0.64	
NaN	Bossons / Blécherette	21	3.26	0.64	
NaN	Montoie / Bourdonnette	22	3.15	0.72	
NaN	Pontaise / Sébeillon	28	3.11	0.75	
NaN	Inconnu	0	NaN	NaN	
NaN					

```
In [45]: # Check available columns before aggregation
print("Available columns in df :", df.columns.tolist())

# Correction : Utilisation de 'RoomAndPropertyType' et 'BasicNightPrice' qui sont présents dans la Ma
# au lieu de 'PropertyType' et 'Price'
try:
    table3 = df.groupby('RoomAndPropertyType').agg(
        nb_annonces=('Airbnb_ListingID', 'count'),
        prix_median=('BasicNightPrice', 'median'),
        taux_occupation_median=('taux_occupation', 'median'),
        revenu_annuel_median=('revenu_annuel_estime', 'median'),
        note_moyenne=('iab_review_scores_rating', 'mean'),
    ).reset_index().sort_values('nb_annonces', ascending=False)

    table3.to_csv('TABLE3_par_type.csv', index=False)
    print("\n✓ TABLE3 : {} types".format(len(table3)))
    display(table3.head(10))
except KeyError as e:
    print(f"\nErreur persistante : {e}. Vérifiez si les colonnes existent bien dans le DataFrame.")
```

Colonnes disponibles dans df : ['Country', 'Location', 'Airbnb_ListingID', 'Scrape_Date', 'Lat', 'Lng', 'MapMarkerType', 'City', 'ListingTitle', 'Bathrooms', 'Bedrooms', 'Beds', 'PersonCapacity', 'is_NewListing', 'PictureCount', 'is_GuestFavourite', 'Host_ID', 'Host_FirstName', 'Host_isSuperhost', 'Host_MultipleListings', 'Host_MultipleListingsGlobal', 'RoomType', 'RoomType_Clean', 'SpaceType', 'SpaceType_Clean', 'RoomAndPropertyType', 'IsHomeOrServicedApartment', 'ReviewCount', 'StarRating', 'RatingPct_5star', 'RatingPct_4star', 'RatingPct_3star', 'RatingPct_2star', 'RatingPct_1star', 'LocationRating', 'CleanlinessRating', 'ValueRating', 'AccuracyRating', 'CheckinRating', 'CommunicationRating', 'VisibleReviewCount', 'IsLowReviewScore', 'NoReviewsNotNew', 'Road', 'Neighbourhood', 'Suburb', 'AddressCity', 'StateDistrict', 'Postcode', 'IndexInSearchEngines', 'BasicNightPrice_CheckIn', 'BasicNightPrice_CheckOut', 'Currency', 'PrimaryPriceString', 'BasicNightPrice', 'BasicNightPrice_USD', 'Taxes', 'Taxes_USD', 'OutlierPrice_PerBedroom', 'OutlierPrice_PerGuest', 'AvailableOverThanTwoWeeks', 'AvailableOver30Days', 'AvailableOver90Days', 'AvailableOver180Days', 'AvailableMoreThan30Days', 'AvailableMoreThan90Days', 'AvailableMoreThan180Days', 'RecordInserted', 'jours_total', 'jours_reserves', 'prix_median_reserve', 'taux_occupation', 'revenu_annuel_estime', 'iab_id', 'iab_review_scores_rating', 'iab_review_scores_cleanliness', 'iab_review_scores_checkin', 'iab_review_scores_communication', 'iab_review_scores_location', 'iab_review_scores_value', 'iab_reviews_per_month', 'iab_number_of_reviews', 'iab_calculated_host_listings_count', 'iab_license', 'iab_match_distance_m', 'quartier_num', 'quartier_nom', 'distance_centre', 'distance_centre_m', 'population', 'nb_logements', 'nb_ménages', 'pct_etrangers', 'densite_hab_km2']

✓ TABLE3 : 31 types

	RoomAndPropertyType	nb_annonces	prix_median	taux_occupation_median	revenu_annuel_median
7	Entire rental unit	557	156.00	0.57	22,545.00
22	Private room in rental unit	207	79.00	0.63	NaN
14	Private room in bed and breakfast	47	80.00	0.60	NaN
20	Private room in home	37	78.00	0.63	NaN
0	Entire apartment	32	169.00	0.64	NaN
5	Entire home	31	274.00	0.68	NaN
25	Private room in villa	10	82.00	0.58	NaN
10	Entire villa	9	537.00	0.76	NaN
26	Room in hotel	8	175.50	0.62	NaN
8	Entire serviced apartment	7	177.00	0.88	NaN

◀ ▶

```
In [46]: # — REF TABLE: Districts (demographics only) —
ref_quartiers.to_csv('REF_quartiers_lausanne.csv', index=False)
print(f"✓ REF : {len(ref_quartiers)} quartiers")
```

✓ REF : 17 quartiers

Part D — Conclusions and Key Results

```
In [47]: # Summary chart: Top districts
fig, axes = plt.subplots(1, 3, figsize=(18, 7))

t2 = table2.sort_values('airbnb_pour_1000_hab', ascending=True)

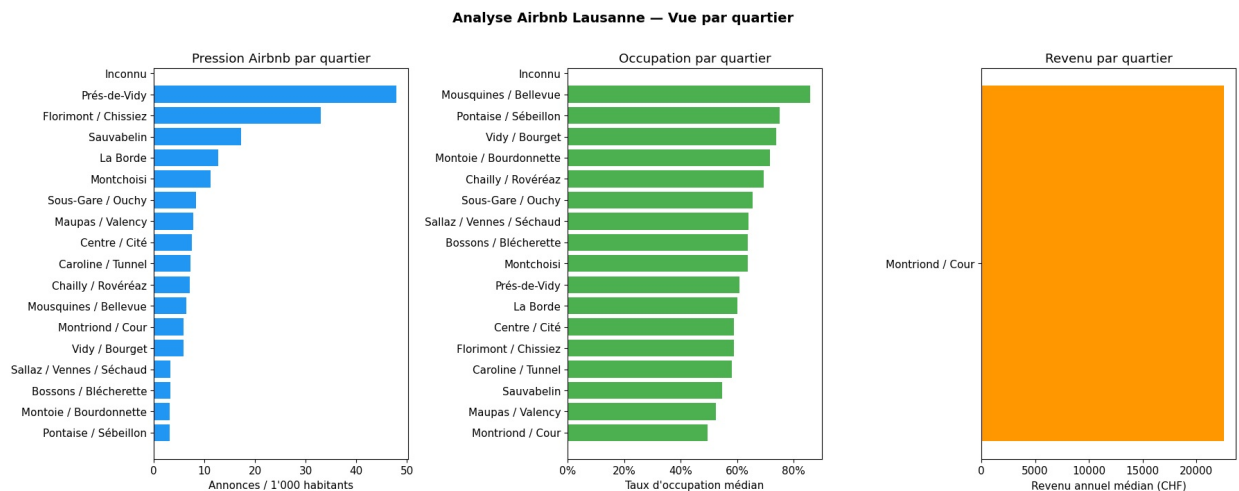
# 1. Densité Airbnb
axes[0].barh(t2['quartier_nom'], t2['airbnb_pour_1000_hab'], color='#2196F3')
axes[0].set_xlabel('Annonces / 1\000 habitants')
axes[0].set_title('Pression Airbnb par quartier')

# 2. Taux d'occupation
t2_occ = table2.sort_values('taux_occupation_median', ascending=True)
axes[1].barh(t2_occ['quartier_nom'], t2_occ['taux_occupation_median'], color='#4CAF50')
axes[1].xaxis.set_major_formatter(mticker.PercentFormatter(1.0))
axes[1].set_xlabel('Taux d'occupation médian')
axes[1].set_title('Occupation par quartier')

# 3. Revenu annuel
t2_rev = table2.sort_values('revenu_annuel_median', ascending=True)
axes[2].barh(t2_rev['quartier_nom'], t2_rev['revenu_annuel_median'], color='#FF9800')
```

```
axes[2].set_xlabel('Revenu annuel médian (CHF)')
axes[2].set_title('Revenu par quartier')

plt.suptitle('Analyse Airbnb Lausanne – Vue par quartier', fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```



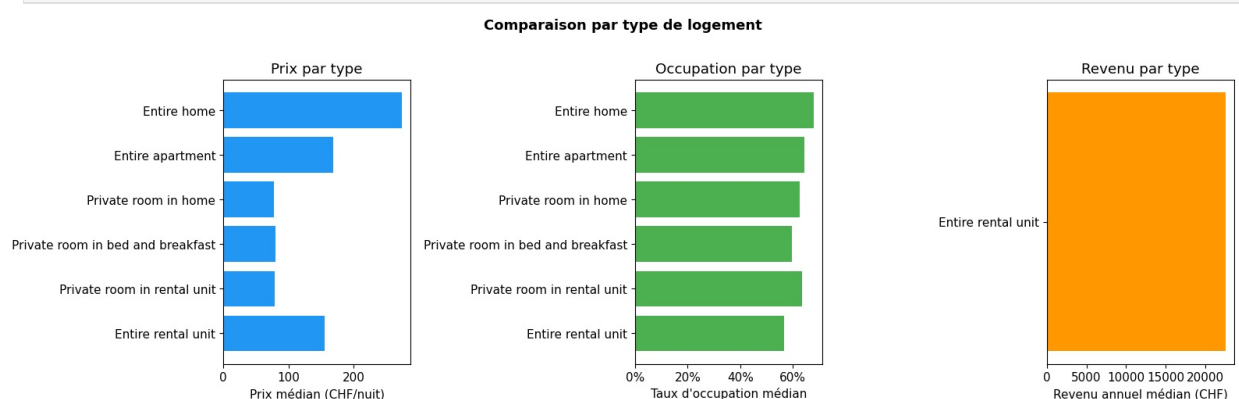
```
In [49]: # Chart: comparison by property type (top 6)
top_types = table3.head(6)
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Utilisation de 'RoomAndPropertyType' au lieu de 'PropertyType'
axes[0].barh(top_types['RoomAndPropertyType'], top_types['prix_median'], color='#2196F3')
axes[0].set_xlabel('Prix médian (CHF/nuit)')
axes[0].set_title('Prix par type')

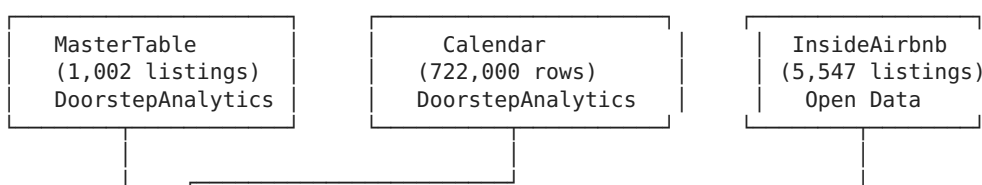
axes[1].barh(top_types['RoomAndPropertyType'], top_types['taux_occupation_median'], color='#4CAF50')
axes[1].xaxis.set_major_formatter(mticker.PercentFormatter(1.0))
axes[1].set_xlabel("Taux d'occupation médian")
axes[1].set_title('Occupation par type')

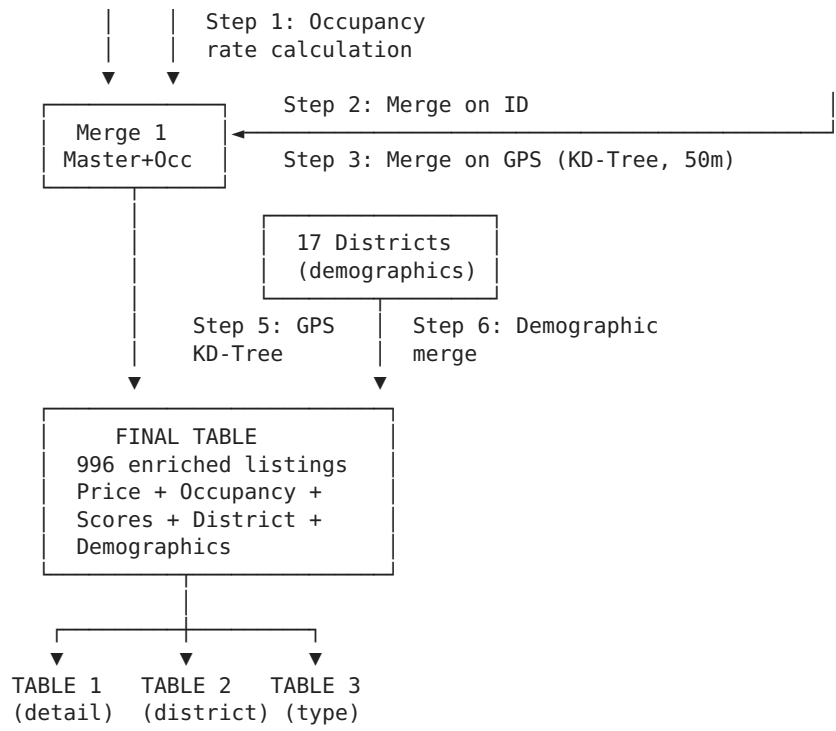
axes[2].barh(top_types['RoomAndPropertyType'], top_types['revenu_annuel_median'], color='#FF9800')
axes[2].set_xlabel('Revenu annuel médian (CHF)')
axes[2].set_title('Revenu par type')

plt.suptitle('Comparaison par type de logement', fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
```



Pipeline Summary

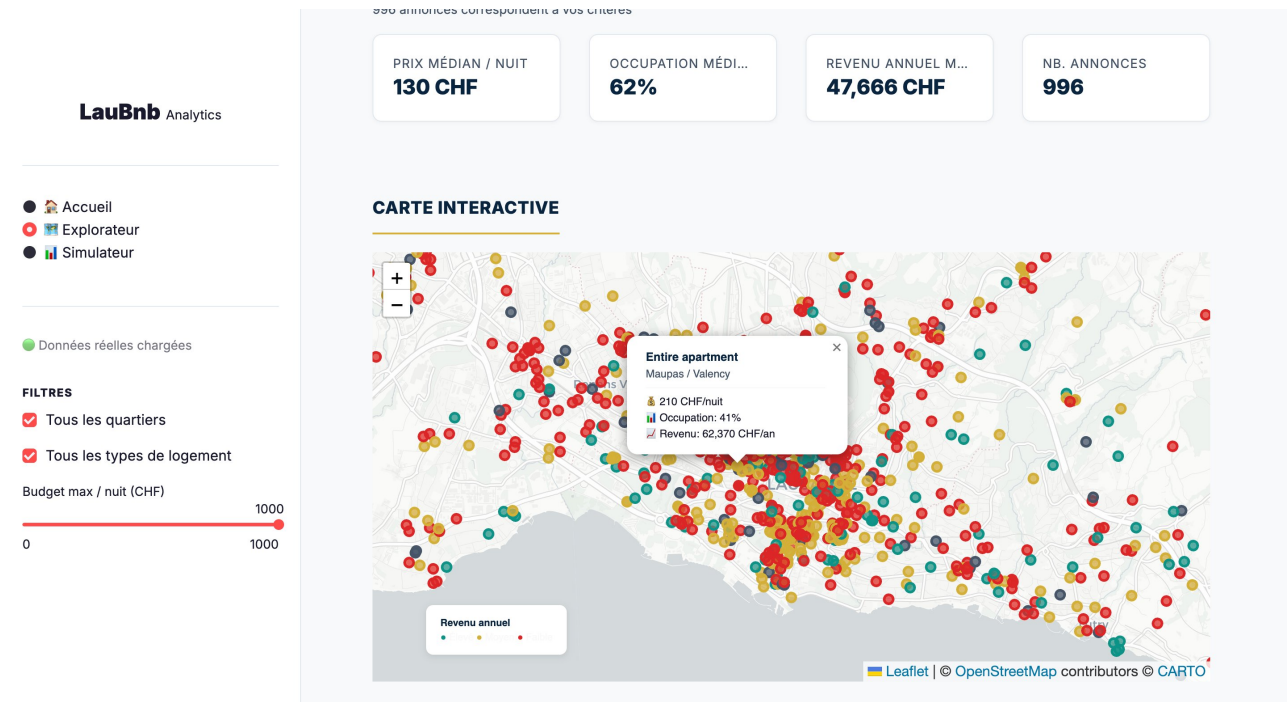




LauBnb Analytics — Delivered Platform

The initial problem was straightforward: individual hosts renting on Airbnb in Lausanne had no way of knowing what price to charge, when to list, or which district to invest in. The data existed, but it was scattered across incompatible sources and unusable as-is. I automated the entire pipeline — collection, cleaning, spatial matching, profitability calculations — and delivered an interactive web platform that turns raw data into actionable decisions. Here is the result.

Homepage — Interactive Map with Real-Time KPIs



LauBnb Analytics — Features

Landing Page — Platform Overview

LauBnb Analytics

● Accueil

● Explorateur

● Simulateur

● Données réelles chargées

Votre **boussole** immobilière

LauBnb Analytics transforme les données du marché Airbnb lausannois en insights actionnables. Identifiez les meilleurs quartiers et simulez votre investissement.

1,000

ANNONCES

17

QUARTIERS

49,231

CHF / AN MÉDIAN

LA PLATEFORME

Données géolocalisées

1'002 annonces rattachées aux 17 quartiers officiels de Lausanne via jointure spatiale KD-Tree.

Indicateurs fiables

Taux d'occupation calculé jour par jour sur 365 jours. Revenus, prix et pression Airbnb par zone.

Simulateur ROI

Testez votre projet : quartier, prix, charges, hypothèque. Résultat en un clic.

1,000 listings across 17 districts — geolocated data via KD-Tree spatial matching.

Investment Simulator — ROI Calculator by District

LauBnb Analytics

● Accueil

● Explorateur

● Simulateur

● Données réelles chargées

Simulateur d'Investissement

Estimez le rendement de votre projet Airbnb à Lausanne

PARAMÈTRES

Quartier visé

Bossons / Blécherette

Données « Bossons / Blécherette »

Revenu médian : 51,370 CHF/an · Occupation : 64% · Prix : 113 CHF/nuit

Prix d'achat (CHF)

500000

Charges mensuelles (CHF)

800

Taux hypothécaire (%)

1.50

Apport personnel (%)

25

RÉSULTATS

RETOUR SUR INVESTISSEMENT NET

7.23%

ROI brut : 10.27% · Verdict : Excellent

REVENU AIRBNB

51,370 CHF

COÛTS ANNUELS

15,225 CHF

REVENU NET

36,146 CHF

↑ positif

RETOUR

13.8 ans

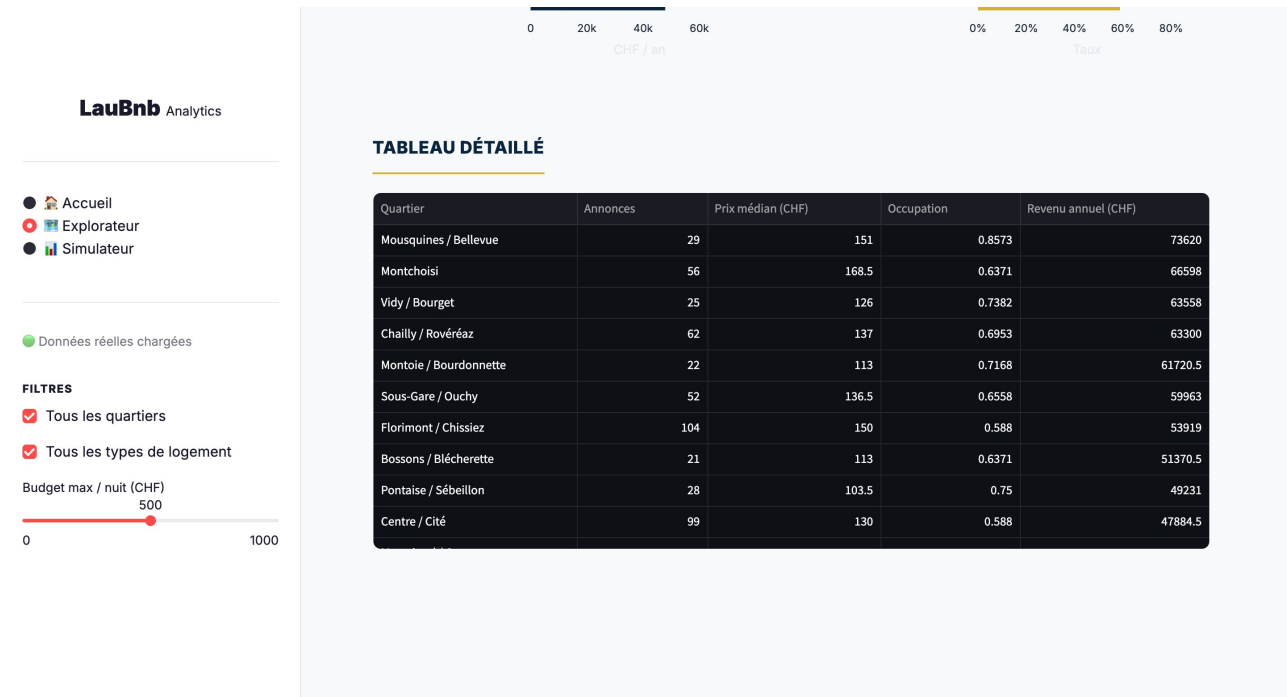
Décomposition du Revenu Net

51,370

Nasser Zenati — Master's Project HES-SO Lausanne 2026

LauBnb Analytics — Detailed Data

Detailed Table — District Rankings



All 17 districts ranked by annual revenue, occupancy rate and median price — fully automated calculations.

Technical Stack

Pipeline: Python, Pandas, NumPy, KD-Tree (scipy)

Sources: Inside Airbnb, availability calendars, official demographic data

Frontend: Streamlit, Leaflet, Plotly

Visualization: Power BI (structured CSV export)

Volume: +1,000 listings × 365 days × 17 districts